

# Capstone Project Instructions

To successfully complete this course and earn your certification, you are required to develop an end-to-end data analytics pipeline, as demonstrated in the video lectures. The project will utilize the Pagila PostgreSQL database ([Database Schema](#)), an alternative to the MySQL Sakila database.

## Project Overview:

1. **Data Source:**
  - You will begin by working with the Pagila transactional database, stored in a PostgreSQL environment. (Credentials [here](#))
2. **Data Replication:**
  - Set up continuous data replication from PostgreSQL to Google BigQuery using Google Datastream. This will ensure that your data remains up-to-date for analysis.
3. **Data Transformation and Analysis:**
  - Utilize BigQuery to perform data transformations and conduct analysis.
  - Supplement these processes with Python notebooks (.ipynb) files that will be converted to python (.py) files, which will handle additional ETL (Extract, Transform, Load) tasks and automation workflows.
  - Notebooks will be developed using Jupyter Notebook locally, reading from our BigQuery data warehouse.
  - Each notebook will produce the corresponding .py executable python file. This is because teams work with python files for code review purposes. Notebooks display many unnecessary lines of code.
  - Python files will be scheduled using **cron** jobs. In a real case scenario, we'll find companies running stuff either using cron or any other workflow orchestration tool such as Prefect, Mage etc.
4. **Version Control:**
  - Use Git to manage your Python .py files and upload them to a GitHub repository, ensuring proper version control and collaboration capabilities.
5. **Data Visualization:**
  - Visualize the processed data using Tableau Cloud and Metabase, enabling you to derive insights and support data-driven decision-making.
  - Publish your data visualizations to your individual Tableau Public and screenshots for Metabase in order to build your portfolio and demonstrate your work.

## Final Deliverables:

A fully functional data analytics pipeline, from data extraction to visualization. You have to create a new github repository and upload everything. You only have to provide the github link to the project submission. It's recommended to set the visibility to public, but if you want to go private mode, consider providing access to [antonisagg@outlook.com](mailto:antonisagg@outlook.com).

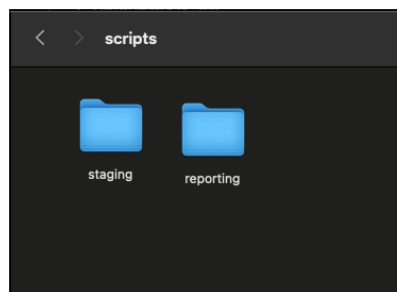
**Regarding the answers for the questions, create a text file and include your answers there.**

\* The candidate has to both create the corresponding visualization and answer the question. Otherwise, no points will be scored.

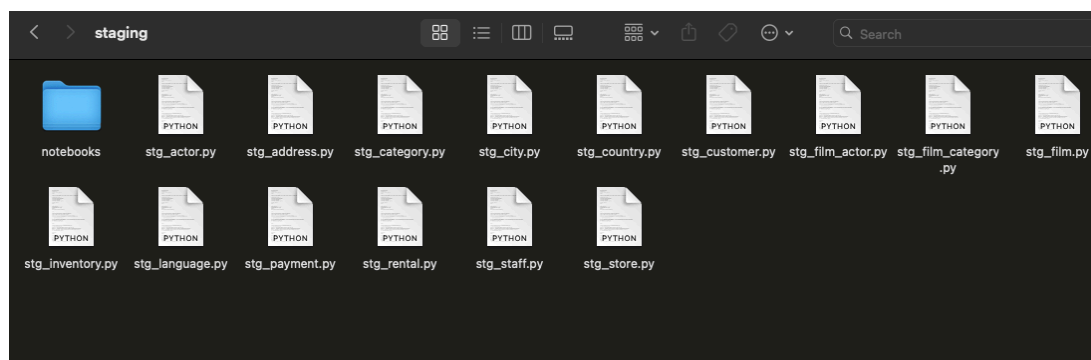
Wrong answers that occur due to minor missing stuff and not impactful errors in the logic, we'll not be taken into account. We want to evaluate the logic.

Your github repo will include:

- 1) Python files for all staging and reporting tables, uploaded to a GitHub repository in your github profile.
  - a) In my example/solution the parent/root folder is named scripts.
  - b) The folder includes 2 sub folders staging (for staging models) and reporting (for reporting models).

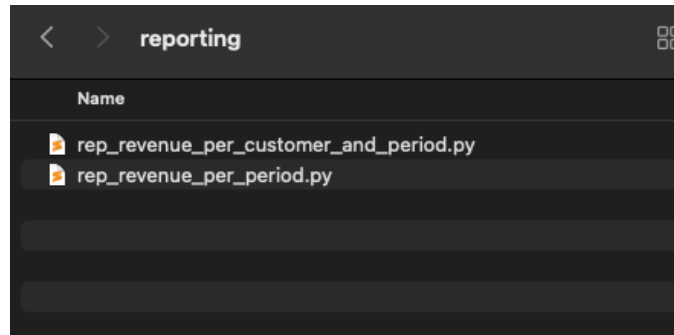


- c) Staging folder includes 15 **.py** scripts, each one creates/updates the corresponding staging table.

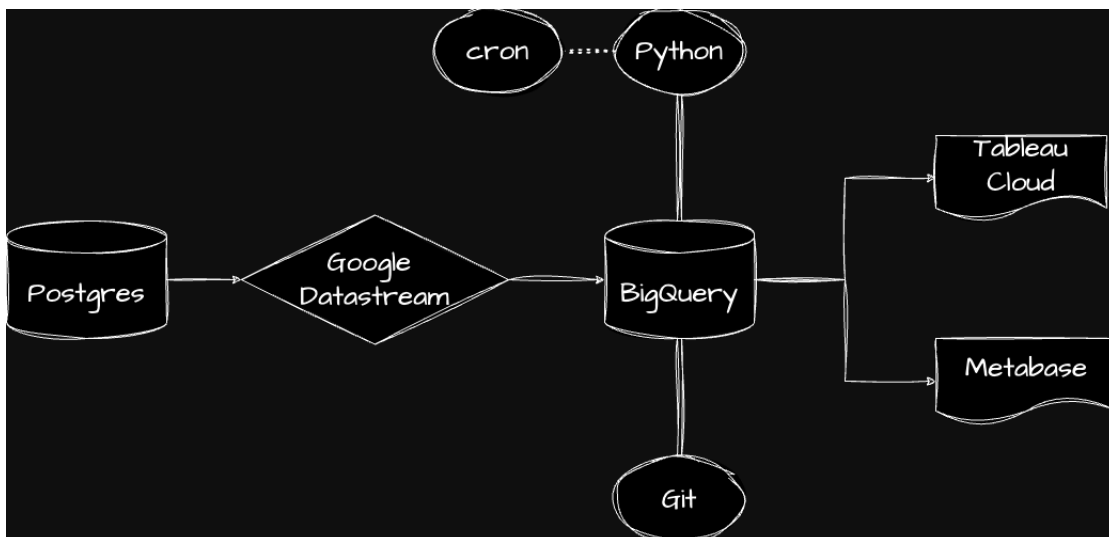


**Important: Including notebooks file is not obligatory**

- d) Reporting folder includes 2 .py scripts for the 2 reporting tables that you need to build.



- 2) Tableau Data visualizations shared on your Tableau Public and Metabase visualizations as screenshots.
- 3) A .txt file with the answers for the questions:
  - a) **Using rep\_revenue\_per\_period**
    - i) What's the % diff between May 22' and June 22'?
    - ii) What's the weekday with the highest average revenue?
  - b) **Using rep\_revenue\_per\_customer\_and\_period**
    - i) Which are the top 5 customers by revenue for June 2022 ?
    - ii) Which is the month with the most unique customers?
- 4) By successfully completing these steps, you will demonstrate your ability to build and manage a comprehensive data analytics pipeline, a critical skill in the field of data analytics.



# 1. Connect to Postgres

To interact with a **PostgreSQL** database, you need a database client. Database clients are software applications designed to facilitate interaction with a Relational Database Management System (RDBMS) like PostgreSQL. They provide user-friendly interfaces—ranging from command-line tools to graphical user interfaces (GUIs)—enabling you to perform various tasks, including:

1. **Creating Databases:** Set up new databases and define their structure.
2. **Executing Queries:** Write and execute SQL queries to retrieve, insert, update, or delete data.
3. **Managing Data:** Perform administrative tasks such as backing up data, restoring databases, and managing users.

Whether you prefer the precision of command-line tools or the ease of visual GUIs, database clients offer the functionality necessary to effectively manage and interact with your PostgreSQL database.

**It's important to note that a PostgreSQL database can serve as the backbone for any application database within a company. From handling transactional data to serving as the primary storage for an enterprise application, PostgreSQL is versatile and reliable enough to meet the needs of various business environments.**

We recommend **beekeeper** ([Installation Guide for Windows](#)) which can be downloaded [here](#) but you can choose **any** client that supports connection with Postgres database.

**Login using credentials that can be found in Section 2 ([here](#))**

Always run your queries using `limit` to reduce performance load.

Example: `select * from customers limit 5`

[Relevant Videos \(2.4.1-2.4.13\)](#)

## 2. Create Google BigQuery (DWH) Instance

**BigQuery** is a fully managed, serverless, and highly scalable data warehouse offered by Google Cloud Platform (GCP) for processing and analyzing large datasets. It allows users to run

fast SQL-like queries on massive datasets using the processing power of Google's infrastructure.

Key Features of BigQuery:

1. **Serverless Architecture:** BigQuery eliminates the need for managing infrastructure such as servers, storage, and networks. It automatically scales to handle your workloads, allowing you to focus solely on analyzing data without worrying about capacity planning.
2. **High Performance:** BigQuery leverages Google's distributed architecture, which enables fast querying and analysis of petabyte-scale datasets. It uses a columnar storage format and can query extremely large datasets in seconds or minutes, depending on the size and complexity of the query.
3. **SQL-Based Interface:** Users interact with BigQuery using SQL, making it accessible to data analysts and engineers familiar with SQL. It also supports standard SQL (ANSI SQL) and has extensions for more advanced features.
4. **Integration with Google Cloud Ecosystem:** BigQuery is deeply integrated with other Google Cloud services like Google Cloud Storage, Google Sheets, and Data Studio, allowing seamless data ingestion, analysis, and visualization. It also integrates with machine learning tools like TensorFlow and BigQuery ML.
5. **BigQuery ML:** BigQuery ML is a feature that allows users to build and train machine learning models directly within BigQuery using SQL queries. This makes it easy for data analysts and business users to create models without extensive coding experience.
6. **Cost-Efficient:** BigQuery uses a pay-as-you-go pricing model where users are charged based on the amount of data processed by queries (measured in bytes) and the amount of data stored. This makes it flexible and cost-efficient for different business needs.
7. **Real-Time Data Analysis:** BigQuery can be used for both batch and real-time data analysis, making it suitable for a wide range of use cases, from reporting and business intelligence to operational analytics and data science.

[Relevant Videos \(6.3.1-6.3.7\)](#)

## BigQuery Data Warehouse Configuration

Create a BigQuery data warehouse using your gmail account. BigQuery is a fully managed, AI-ready data platform that helps you manage and analyze your data with built-in features like machine learning, search, geospatial analysis, and business intelligence. BigQuery's serverless architecture lets you use languages like SQL and Python to answer your organization's biggest questions with zero infrastructure management.

**Important:** Google offers \$300 in free credits to new customers to try Google Cloud products and build a proof of concept. Credit amount and period are sufficient for this course needs, you will have to submit your project before the expiration date. You won't be charged until you activate your full paid account. There is no charge to use these products up to their specified [free usage limit](#). The free usage limit does not expire, but is subject to change. In the end, you will have consolidated and transformed data ready for analysis. Free credits will be also used for Google Datastream which will replicate the data from Postgres production database to BigQuery.

[Relevant Videos \(6.4\)](#)

### 3. Create Google Datastream Connection

[Google Datastream](#) is a fully managed, serverless data replication and change data capture (CDC) service offered by Google Cloud. It is used to synchronize data in real-time between operational databases (like MySQL, PostgreSQL, or Oracle) and data warehouses, such as BigQuery, or other analytics platforms.

Key Features of Google Datastream:

1. **Change Data Capture (CDC):** Datastream captures changes to data in real-time from supported databases. It allows organizations to track inserts, updates, and deletes happening in a source database and replicate them to a target data system like BigQuery, Google Cloud Storage, or Cloud Spanner.
2. **Serverless and Fully Managed:** Like many Google Cloud services, Datastream is serverless, which means it abstracts away infrastructure management. You don't need to worry about provisioning or maintaining servers, and it scales automatically to meet the demands of your workload.
3. **Real-Time Data Replication:** It enables real-time replication of data, ensuring that the data in the target system (such as BigQuery or Cloud SQL) is always up to date. This is crucial for use cases like operational analytics, reporting, and real-time dashboards.
4. **Event-Driven Architecture:** Google Datastream allows event-based architecture by capturing database changes as they occur. The changes are streamed as events, which can trigger downstream systems or workflows, making it useful for real-time applications.
5. **Schema Evolution:** Datastream handles schema changes gracefully, allowing you to manage and synchronize schema updates between the source and the destination without needing to pause the data flow.
6. **Data Flow Customization:** Google Datastream integrates with **Dataflow**, another GCP service, allowing you to transform or enrich the data before it reaches its destination. This flexibility ensures that your replicated data meets your specific needs.
7. **Integration with Google Cloud Services:** Datastream works well with other GCP services, like:
  - **BigQuery:** For real-time analytics and data warehousing.

- **Google Cloud Storage:** For storing raw data or using it as a landing zone for subsequent processing.
  - **Cloud SQL:** For replicating data into relational databases in Google Cloud.
  - **Dataflow:** For real-time stream processing or transforming data in-flight.
8. **High Availability and Reliability:** Datastream provides automatic fault tolerance, and since it's a managed service, Google Cloud ensures high availability and reliability for data replication tasks.
9. **Source and Target Support:**
- **Sources:** Currently, Datastream supports databases like MySQL, PostgreSQL, and Oracle for capturing changes in real time.
  - **Targets:** Common targets include BigQuery, Google Cloud Storage, and Cloud SQL, but it can integrate with other systems via connectors or pipelines.

## Google Datastream Configuration

Functional data streaming setup from PostgreSQL to BigQuery. (Using \$300 free credits). Datastream provides seamless replication of data from operational databases into BigQuery. In addition, Datastream supports writing the change event stream into Cloud Storage, and offers streamlined integration with Dataflow templates to build custom workflows for loading data into a wide range of destinations, such as Cloud SQL and Spanner. You can also use Datastream to leverage the event stream directly from Cloud Storage to realize event-driven architectures. Datastream supports Oracle, MySQL, SQL Server, and PostgreSQL (including AlloyDB for PostgreSQL) sources.

[Relevant Videos \(6.5.1-6.5.9\)](#)

[GCP Service Account Roles](#)

## Important

1. You should run the below SQL commands right before pressing the final create and start stream button. (with a max 2–3 mins difference). The below 2 commands ensure that your replication slot is ready and waits for a new stream.

```
select pg_create_logical_replication_slot('student_replication', 'pgoutput');
```

```
select pg_drop_replication_slot('student_replication');
```

2. **Remember to Pause the Stream** in order not to replicate the data repeatedly and burn the free Google credits. In this course, we'll only replicate the tables once.

## 4. Create staging tables

This course structures the BigQuery tables and data warehouse schemas and logic, **based on dbt recommendations**, although **dbt is not used** in the final project for two reasons. It demands some prerequisite advanced skills and it's not what this course is all about.

### Overview of DBT Structure

dbt(data build tool) is a popular analytics engineering tool that helps data teams transform raw data into an organized, analytics-ready format. The structure of a dbt project is essential for managing complexity and improving the scalability of the data transformation pipeline. Here's an overview of the recommended structure for dbt projects, broken into stages, intermediate, and final models.

The dbt project is typically organized into the following layers:

1. Staging Layer
2. Intermediate Layer
3. Reporting Layer

Each layer serves a specific function in transforming data from its raw form into the final state that can be used for reporting and analysis.

### Why This Structure?

The key motivation behind structuring a dbt project in layers is to:

1. Improve readability and maintainability.
2. Clearly separate raw data from transformed data.
3. Enable easier debugging and reusability of transformations.

### Staging Layer

The staging layer serves as the initial layer of transformations that interact directly with your raw data. The purpose of this layer is to create clean, standardized tables that act as a foundation for further transformations. Some of the common practices and goals of this layer are:

1. Raw data preparation: Data from source systems (like databases, APIs, etc.) is brought into staging tables.
2. Minimal transformation: The primary focus is on renaming columns, handling missing or incorrect values, and standardizing formats.



3. One source per table: Each staging table represents data from a single source table, so it's a one-to-one mapping.
4. No business logic: At this stage, you are not applying any business rules or calculations—just preparing data for further transformation.

Each file in the staging layer typically reflects a single source and applies basic cleaning or formatting.

### Intermediate Layer

The intermediate layer sits between staging and the final data models, serving as a bridge for more complex transformations. This is where business logic starts to be applied. Common goals of this layer include:

1. Join tables: Combine data from multiple staging tables to create enriched datasets that contain data from various sources.
2. Apply business logic: Incorporate transformations like calculations, data aggregations, or mappings that follow specific business rules.
3. Prepare data for final reporting: This layer transforms clean data from the staging layer into more useful structures that are ready for reporting or further modeling.

In this layer, multiple staging tables might be combined (e.g., joining orders with customer data), and complex transformations start to take place.

### Reporting (Mart) Layer

While not always explicitly mentioned, dbt projects often end with a reporting layer where data is shaped into the final reporting-ready format. This is where metrics, KPIs, and final aggregations are defined. Typically, this layer contains the data marts or fact tables that business users directly interact with for analytics and reporting.

**In this course, we'll not use the intermediate layer, as it would require even more effort and complexity exceeding course's length.**

### Modern Data Structure:

```
models/ └─ staging/
              └─ stg_customer..sql
            / └─ intermediate/
                  └─ int_customer_revenue_per_period.sql
            / └─ reporting/
                  └─ rep_revenue_per_period.sql
```

**Our structure will be like:**

```
models/ └─ staging/
              └─ stg_customer..sql
            /─ intermediate/
              └─ int_customer_revenue_per_period.sql
          /─ reporting/
            └─ rep_revenue_per_period.sql
```

### **Objective:**

You will create a schema called `staging_db` and then create corresponding staging tables for each production table

### **Example**

customers → stg\_customer

users → stg\_users

student → stg\_student

**Each staging table should be created following the specific requirements below.**

## 4.1 Step 1: Create the staging\_db Schema

Start by creating the schema `staging_db` in your PostgreSQL database

## 4.2 Step 2: Create Corresponding Staging Tables

For each production table in your database, you will create a corresponding staging table within the `staging_db` schema. The structure of each staging table should reflect the transformations and data processing steps outlined in the respective Python notebooks.

Follow the samples [here \(stg\\_actor.zip\)](#).

## 4.3 General Guidelines

- **Table Naming Convention:**

Use the prefix `stg_` followed by the name of the corresponding production table. For example, if your production table is named `customer`, the staging table should be named `stg_customer`.

- **Data Types:**

The data types of the columns in the staging tables should match those in the production tables, unless transformations in the Python notebooks dictate otherwise.

[Relevant Videos \(6.6.1-6.6.3\)](#)

## 4.4 Staging tables

The staging layer is where our journey begins. This is the foundation of our project, where we bring all the individual components we're going to use to build our more complex and useful models into the project. Staging models act as the very first contact point for any Intermediate Model or Marts model. As there are very basic operations or data transformations that have to be done as early in the data transformation pipeline as possible, e.g. translating French columns into English, timezone conversion, id from table user to user\_id etc. Using Pagila database, you have to create a corresponding staging table for every table that exists in the database based on the description below. You'll create a new python notebook .ipynb file that will be used to create a specific table.

**Example:** The script `stg_customer.ipynb` will create the table `stg_customer` which will be generated from the raw table `customer`. We'll then use the python `stg_customer.py` to update our data warehouse. Remember, that you have to convert .ipynb to .py python executable file. Follow the samples [here \(stg\\_actor.zip\)](#) script as your base and modify the code accordingly only between below lines. This course **does not** focus on the python fundamentals.

You only have to modify code which is below and above the lines indicating where to edit your code.

```
# -- YOUR CODE GOES BELOW THIS LINE

your_code_is_here

# -- YOUR CODE GOES ABOVE THIS LINE
```

|         |                                                                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| actor   | Add prefix <code>actor_</code> to columns: <code>first_name</code> , <code>last_name</code> , <code>last_update</code>                                               |
| address | Add prefix <code>address_</code> to columns: <code>district</code> , <code>city_id</code> , <code>postal_code</code> , <code>phone</code> , <code>last_update</code> |

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| category      | Add prefix <code>category_</code> to columns: <code>name</code> , <code>last_update</code>                                                                                                                                                                                                                                                                                                                                                                        |
| city          | Add prefix <code>city_</code> to columns: <code>last_update</code><br>Rename <code>city</code> to <code>city_name</code>                                                                                                                                                                                                                                                                                                                                          |
| country       | Add prefix <code>country_</code> to columns: <code>last_update</code><br>Rename <code>country</code> to <code>country_name</code>                                                                                                                                                                                                                                                                                                                                 |
| customer      | Add prefix <code>customer_</code> to columns: <code>store_id</code> , <code>first_name</code> ,<br><code>last_name</code> , <code>email</code> , <code>address_id</code> , <code>create_date</code> , <code>last_update</code><br>Rename <code>activebool</code> to <code>is_active_customer_bool</code><br>Rename <code>active</code> to <code>is_active_customer</code>                                                                                         |
| film          | Add prefix <code>film_</code> to columns: <code>title</code> , <code>description</code> , <code>language_id</code> ,<br><code>original_language_id</code> , <code>rental_duration</code> , <code>rental_rate</code> ,<br><code>length</code> , <code>replacement_cost</code> , <code>rating</code> , <code>last_update</code> ,<br><code>special_features</code> , <code>fulltext</code><br>SchemaField for <code>special_features</code> = <code>'STRING'</code> |
| film_actor    | Add prefix <code>film_actor</code> to columns: <code>last_update</code>                                                                                                                                                                                                                                                                                                                                                                                           |
| film_category | Add prefix <code>film_category</code> to columns: <code>last_update</code>                                                                                                                                                                                                                                                                                                                                                                                        |
| inventory     | Add prefix <code>inventory_</code> to columns: <code>last_update</code>                                                                                                                                                                                                                                                                                                                                                                                           |
| language      | Add prefix <code>language_</code> to columns: <code>name</code> , <code>last_update</code>                                                                                                                                                                                                                                                                                                                                                                        |
| payment       | Add prefix <code>payment_</code> to columns: <code>amount</code>                                                                                                                                                                                                                                                                                                                                                                                                  |
| rental        | Add prefix <code>rental_</code> to columns: <code>return_date</code> , <code>staff_id</code> , <code>last_update</code>                                                                                                                                                                                                                                                                                                                                           |
| staff         | Add prefix <code>staff_</code> to columns: <code>first_name</code> , <code>last_name</code> , <code>address_id</code> ,<br><code>email</code> , <code>username</code> , <code>password</code> , <code>last_update</code> , <code>picture</code><br>Rename <code>active</code> to <code>is_active_staff</code><br>SchemaField for <code>picture</code> = <code>'BYTES'</code>                                                                                      |
| store         | Add prefix <code>store_</code> to columns: <code>address_id</code> , <code>last_update</code>                                                                                                                                                                                                                                                                                                                                                                     |

## 4.5 Final Step

- **Link to Python Notebooks:**  
Ensure that each staging table creation is associated with the corresponding Python notebook that documents the transformations and logic applied.
- **Upload to GitHub:**  
Once all staging tables are created, document your SQL scripts and Python notebooks, and upload them to the designated GitHub repository.

## 4.5 Setup Jupyter & Git

This [guide](#) includes all necessary information to:

1. Install Jupyter [Relevant Videos \(6.7.1-6.7.5\)](#)
2. Connect to BigQuery locally using .json key for a service account with the necessary permissions [Relevant Videos \(6.8.1-6.8.3\)](#)
3. Create the staging tables [Relevant Videos \(6.9.1-6.9.15\)](#)
4. Install and configure Git [Relevant Videos \(6.10.1-6.10.6\)](#) ([Local Git with github pass](#))

## 5. Create reporting tables

### Objective:

Using the staging tables created earlier, you will create two reporting tables that will be utilized for data analysis in Tableau Public and Metabase.

You have to reproduce the tables `all_dates` and `reporting_periods` as described in the video lectures as they are prerequisites for the following requested tables.

[Relevant Videos \(6.11.1-6.11.6\)](#)

The tables that you have to create should adhere to the following guidelines:

### 5.1 Reporting Table 1: rep\_revenue\_per\_period

#### Purpose:

This table will contain revenue data aggregated by different periods (Day, Month, Year) for analysis.

#### Requirements:

- **Period Coverage:**
  - The table should cover periods from `'2015-01-01'` to the latest date available in the `reporting_periods_table`. (No restrictions here, just include at least 2022, 2023, 2024).
  - Include periods `'Day'`, `'Month'`, `'Year'`.
- **Revenue Inclusion:**
  - Ensure that records are included even if the revenue for the corresponding period equals `0`.
  - For rows in which `total_revenue` is null, convert null to 0.
- **Exclusion Criteria:**
  - Excludes film with title `'GOODFELLAS SALUTE'` which is not billed and offered for free

#### Table Structure Suggestion:

- `reporting_period`: (e.g., 'Day', 'Month', 'Year')
- `reporting_date`: (The specific date of the period)
- `total_revenue`: (Revenue for the period, `0` if no revenue)

#### [Relevant Videos \(6.12.1-6.12.9\)](#)

## 5.2 Reporting Table 2: `rep_revenue_per_customer_and_period`

### Purpose:

This table will provide detailed revenue data, broken down by customer and period.

### Requirements:

- **Period Coverage:**
  - Include periods `'Day'`, `'Month'`, `'Year'` for all the dates that make sense. Here, we only need records for dates in which there's any revenue collected.
- **Exclusion Criteria:**
  - Excludes film with title `'GOODFELLAS SALUTE'` which is not billed and offered for free

#### Table Structure Suggestion:

- `customer_id`: (Unique identifier for the customer)
- `reporting_period`: (e.g., 'Day', 'Month', 'Year')
- `reporting_date`: (The specific date of the period)
- `total_revenue`: (Revenue for the customer for the specific period and date)

[Relevant Videos \(6.12.10-6.12.15\)](#)

## 5.4 Final Step

- **Upload to GitHub:**

Once the tables are created, ensure your SQL scripts or Python files from the python notebooks are properly documented and upload them to the designated GitHub repository.

[Relevant Videos \(6.10.1-6.10.6\) \(Local Git with github pass\)](#)

## 6. Workflow Orchestration and Jobs

Python scripts will be scheduled using cron jobs. A cron job is a Linux command used for scheduling tasks to be executed sometime in the future. This is normally used to schedule a job that is executed periodically. In the real world,, we'll find companies running stuff either using cron or any other workflow orchestration tool such as [Prefect](#), [Mage](#) etc. A cron expression is a string consisting of six or seven subexpressions (fields) that describe individual details of the schedule. Read [here](#) for cron expressions.

Here, the cron job orchestration will run on the participant's PC. In a real case scenario, this PC is a company's property.

**Important:** The course will not focus on the job/workflow orchestration part, thus it will not be evaluated.

**Windows users will set up CRON using linux cmd commands for windows with WSL (recommended) or by following [this](#) guide and [this](#) video.**

**MacOS/Linux**

### **Steps to Use Nano to Edit Crontab (MACos)**

Open your terminal. Set Nano as the default editor for crontab by running:

```
export EDITOR="nano"
```

1. To access and edit the crontab file, open Terminal and type the following command:  
`crontab -l`. This will list all cron jobs and if runs successfully it means that cron is installed in your system.
2. Make a new `.sh` file named `run_staging.sh` that will be triggered from cron to execute the `.py` scripts using a text editor like Sublime Text or Nano. [Here](#), you can find a sample that worked in the instructor's setup.
3. By default, cron doesn't set up the PATH environment variable the same as a normal login shell so python may not be found. It should work if you specify a complete path to the Python executable, e.g. replace python with `/usr/bin/python` (or whatever the path to your Python executable happens to be). Alternatively you can explicitly set the PATH environment variable in the Cron configuration file to include the directory where Python can be found. Read more [here](#).
4. Make the script executable with the `chmod` command (by navigating with `cd` terminal command to your local directory which `run_staging.sh` is saved:

```
chmod +x run_staging.sh
```

5. Create/Edit crontab File

Open the crontab file:

```
crontab -e
```

The file opens in the vi editor. If the crontab had not previously existed, a new file is created. Add a line containing a Cron expression and the path to the script. A Cron expression consists of five fields that accept the respective number values:

For example, to create an expression that executes the `run_staging.sh` located in your home directory daily at 20:36 p.m., (press i to turn on the Insert Mode and add the following line to the crontab file, or paste from editor):

```
36 20 * * * /Users/antonesangelakes/scripts/run_staging.sh
>> /Users/antonesangelakes/scripts/cronjob.log 2>&1
```

Don't miss on editing the minutes and hour of job run.

You now should wait and revisit/open the `cronjob.log` to validate the successful execution.



-- BELOW IS OPTIONAL--

Some example code

# Backup script daily at 3 AM

```
0 3 * * * /path/to/backup.sh
```

# Run a Node.js script every hour

```
0 * * * * /usr/local/bin/node /path/to/script.js
```

# Run a Python script every Monday at 4 PM

```
0 16 * * 1 /usr/bin/python3 /path/to/script.py
```

# Runscript every 2 minutes

```
*/2 * * * * /path/to/backup.sh
```

We can also add a `#` before every expression if we want to disable it. It's like commenting out.

6. If the cron job does not run, this may happen due to insufficient privileges of disk access.

[Granting Full Disk Access to cron](#)

[How to Fix Cron Permission Issues in macOS Big Sur, Catalina, & Mojave](#)

[Crontab operation not permitted](#)

7. Sometimes, we might need to log the cron job output on stdout or any other log file for monitoring and maintenance purposes. Fortunately, we have several methods that we can use to log the cron job output.

**It's recommended** to create a log file that will include any returned error message.

```
*/2 * * * *
/Users/firstname_lastname/Documents/run_staging.sh
>/Users/firstname_lastname/Documents/stdout.log
2>/Users/firstname_lastname/Documents/stderr.log
```

[Redirecting the Output of a Cron Job to stdout](#)

Finally, this is a [run\\_staging.sh](#) script that schedules a python script. Modify accordingly for your own system.

[Relevant Videos \(cron-1-cron-3\)](#)

## 7. Visualize in Tableau Public & Metabase

### 7.1 Tableau Public

Create a [Tableau Public profile](#) and upload your visualizations in your profile with the free community version of Tableau Desktop the [Tableau Public Desktop](#). You have to export the tables as .csv files and use these as data sources. Tableau Public Desktop does not support direct BigQuery connection.

[Relevant Videos \(6.13.1 - 6.13.7\)](#)

### 7.2 Metabase

We will use and run Metabase locally to create our visualizations. Metabase offers the capability to directly establish a BigQuery connection and pull the data.

Visit [Metabase Open Source](#). We will install Metabase using [Docker](#) and we will run it on localhost.

#### MacOS:

1. Update Homebrew to the latest version:  
`brew update`
2. Run  
`brew install --cask docker`
3. Verify Docker installation:  
`docker --version`
4. You will then need to open Docker from Applications or via Terminal:  
`open /Applications/Docker.app`
5. Since Docker Desktop has opened, you can now install Metabase by running:  
`docker run -d -p 3000:3000 --name metabase metabase/metabase`
6. Visit <http://localhost:3000/> and fill in your user information. Connect to your BigQuery instance using your GCP service account JSON file and voila!

You have successfully connected BigQuery with Metabase like any professional environment!

#### Windows:

1. Install [WSL](#).
2. Install [Docker](#) for Windows.
3. Open Docker Desktop
4. Since Docker Desktop has opened, you can now install Metabase by running:

`docker run -d -p 3000:3000 --name metabase metabase/metabase` from the WSL Interface.

5. Visit <http://localhost:3000/> and fill in your user information. Connect to your BigQuery instance using your GCP service account JSON file and voila!

[Relevant Videos \(6.14.1 - 6.14.11\)](#)

## 7.3 Specifications

- Using **rep\_revenue\_per\_period** you have to create:
  - A line chart visualizing the revenue per date by providing the capability to choose the date period and filter the dates.
    - Question: What's the % diff between May 22' and June 22'?
  - A bar chart visualizing the average revenue per weekday.
    - Question : What's the weekday with the highest average revenue?
- Using **rep\_revenue\_per\_customer\_and\_period** you have to create:
  - A table which will include top 5 customers per period, by providing the capability to choose the date period and filter the dates.
    - Question: Which are the top 5 customers by revenue for June 2022 ?
  - A bar chart which will visualize the number of unique customers with >0 revenue per month.
    - Question: Which is the month with the most unique customers?

Tableau Public: URL of Tableau Public Dashboard

Metabase: Screenshots

## 8. Capstone Project Evaluation

Your Capstone Project will be evaluated based on the following criteria:

## 8.1 Data Integration (20/100)

- Effectiveness and Efficiency of Data Streaming and Synchronization:

**Task:** You must successfully establish a data stream between a Postgres database and your BigQuery instance.

## 8.2 Data Transformation and Upload to Git (40/100)

- Quality and Accuracy of Data Transformations and Aggregations: Use Python Notebooks for ETL (Extract, Transform, Load) and automation tasks.
  - Create all 15 corresponding staging tables in the `staging_db` schema. (Each table counts for 2 points)
  - Create `rep_revenue_per_period` and `rep_revenue_per_customer_and_period` in the `reporting_db` schema. (Each table counts for 5 points)
  - Create a separate `.py` python script for each table.
- Project Code Management: Demonstrate proper use of Git for managing your project code.

**Task:** Upload all Python scripts that create the above tables to your GitHub profile.

## 8.3 Visualization (40/100)

- Quality, Interactivity, and Insights Provided by Dashboards:

**Tasks:** Create 1 Tableau Dashboard using Tableau Public including the following visualizations.

- Using `rep_revenue_per_period` you have to create:
  - A line chart visualizing the revenue per date by providing the capability to choose the date period and filter the dates. Question: What's the % diff between May 22' and June 22'?
  - A bar chart visualizing the average revenue per weekday. Question : What's the weekday with the highest average revenue?
- Using `rep_revenue_per_customer_and_period` you have to create:
  - A table which will include top 5 customers per period, by providing the capability to choose the date period and filter the dates. Question: Which are the top 5 customers by revenue for June 2022 ?
  - A bar chart which will visualize the number of unique customers with >0 revenue per month. Question: Which is the month with the most unique customers?

**Tasks:** Create Metabase visualizations to answer the questions **WITHOUT** the dashboard functionality.

\* The candidate has to both create the corresponding visualization and answer the question. Otherwise, no points will be scored. Each visualization is worth 5 points, contributing to a total of 40 points for this section. Wrong answers that occur due to minor missing stuff and not impactful errors in the logic, we'll not be taken into account. We want to evaluate the logic.